

实体克隆（Entity Cloning）

1. 实体克隆简介

Bevy 现在正式支持实体（entity）的克隆功能。虽然在此之前也可以通过反射（Reflect）来实现克隆，但这种通用实现方式速度较慢，而且要求提前注册所有可克隆的组件。自 Bevy 0.16 起，实体克隆已由引擎原生支持，只需在组件上添加 `#[derive(Clone)]` 即可将其设为可克隆。

```
#[derive(Component, Clone)]
#[require(Health(100.0), Attack(10.0))]
struct Enemy;

#[derive(Component, Clone)]
struct Health(f32);

#[derive(Component, Clone)]
struct Attack(f32);

#[derive(Component, Clone)]
struct Weapon;
```

2. 实体克隆方法

- `clone_and_spawn`: 会创建一个新实体，并复制原实体中所有可克隆的组件，跳过那些不可克隆的组件。
- `clone_components`: 将组件从源实体克隆到指定目标实体，而不是生成一个新实体。
- `clone_and_spawn_with` 与 `clone_with`: 提供了对 `EntityClonerBuilder` 的访问，可以用来自定义克隆行为。
- `EntityClonerBuilder`: 允许你配置克隆逻辑，例如过滤哪些组件应该被克隆、定制特定组件的克隆方式，或控制关系实体是否递归克隆。
- `move_components`: 在克隆组件到目标实体之后，从源实体移除这些组件。

```
// 克隆敌人
fn clone_enemies(mut commands: Commands, enemies: Query<Entity, With<Enemy>>()) {
    for entity in enemies {
        // 将此实体中的可克隆组件，克隆到新生成的实体中
        commands.entity(entity).clone_and_spawn();

        let target = commands.spawn_empty().id();
        // 将指定组件克隆到目标实体
        commands
            .entity(entity)
            .clone_components::<(Enemy, Health, Attack)>(target);

        // 克隆到新实体并配置克隆行为
        commands.entity(entity).clone_and_spawn_with(|builder| {});

        let target = commands.spawn_empty().id();
        // 克隆到目标实体并配置克隆行为
        commands.entity(entity).clone_with(target, |builder| {});

        let target = commands.spawn_empty().id();
        // 将指定组件移动到目标实体中
        commands
            .entity(entity)
            .move_components::<(Enemy, Health, Attack)>(target);
    }
}
```

3. 配置克隆逻辑

你可以使用如下的 `EntityClonerBuilder` 方法配置克隆逻辑：

- `deny_all`: 所有组件都不允许克隆
- `deny`: 不允许克隆指定组件
- `allow_all`: 允许克隆所有可克隆组件
- `allow`: 允许克隆指定组件
- `linked_cloning`: 可以克隆子实体
- `move_components`: 将克隆变成移动
- `add_observers`: 如果被克隆的实体在观察者的观察列表中，那么就将新实体也添加到观察者的观察列表中

```
// 克隆到新实体并配置克隆行为
commands.entity(entity).clone_and_spawn_with(|builder| {
    builder
        // 所有组件都不允许克隆
        .deny_all()
        // 允许克隆 Enemy
        .allow::<Enemy>()
        // 可以克隆子实体
        .linked_cloning(true); // 默认为 false
});

let target = commands.spawn_empty().id();
// 克隆到目标实体并配置克隆行为
commands.entity(entity).clone_with(target, |builder| {
    builder
        // 将克隆变成移动
        .move_components(true) // 默认为 false
        // 不允许克隆 Health
        .deny::<Health>()
        // 允许克隆所有可克隆组件
        .allow_all()
        // 如果被克隆的实体在观察者的观察列表中，那么就将新实体也添加到观察者的观察列表中
        .add_observers(true); // 默认为 false
});
```

4. 注意事项

带有泛型类型参数的组件默认不能被克隆。在这种情况下，你需要为该组件添加 `#[derive(Reflect)]` 和 `#[reflect(Component)]` 属性，并在程序中注册它，才能正常使用实体克隆功能。

```
App::new()
    // 注册泛型组件
    .register_type::<GenericComponent<i32>>()
    .run();

// 泛型组件必须实现反射，才能被克隆
#[derive(Component, Reflect, Clone)]
#[reflect(Component)]
struct GenericComponent<T> {
    value: T,
}

// 克隆泛型组件
fn clone_generic_components(
    mut commands: Commands,
    generic_components: Query<Entity, With<GenericComponent<i32>>>,
) {
    for entity in generic_components {
        commands.entity(entity).clone_and_spawn();
    }
}
```